

gDifferent Types of SQL JOINS

Join Type	Description	Example Syntax where A and B is a database table
INNER JOIN	Returns only matching rows from both tables.	SELECT * FROM A INNER JOIN B ON A.id = B.a_id;
LEFT JOIN	Returns all rows from the left table, and matched rows from the right.	SELECT * FROM A LEFT JOIN B ON A.id = B.a_id;
RIGHT JOIN	Returns all rows from the right table, and matched rows from the left.	SELECT * FROM A RIGHT JOIN B ON A.id = B.a_id;
FULL JOIN	Not directly supported in MySQL, but can be simulated using UNION. Returns all records when there is a match in either left or right table	SELECT * FROM A LEFT JOIN B ... UNION SELECT * FROM A RIGHT JOIN B ...
CROSS JOIN	Returns the Cartesian product of both tables (every combination).	SELECT * FROM A CROSS JOIN B;
SELF JOIN	Joins a table to itself, useful for hierarchical data.	SELECT * FROM A a1 JOIN A a2 ON a1.manager_id = a2.id;

Example:

The screenshot shows a database management interface. At the top, a breadcrumb path indicates the current location: **Server: 127.0.0.1 » Database: mydb » Table: departments**. Below this is a toolbar with icons for **Browse**, **Structure**, **SQL**, **Search**, **Insert**, **Export**, and **Import**. A green status bar reports: **Showing rows 0 - 2 (3 total, Query took 0.0002 seconds.)**. The SQL query entered is `SELECT * FROM `departments``. Below the query, there are links for **Profiling**, **Edit inline**, **Edit**, **Explain SQL**, **Create PHP code**, and **Refresh**. A control bar shows **Show all**, **Number of rows: 25**, **Filter rows: Search this table**, and **Sort by key:**. An **Extra options** button is also present. The main area displays a table with two columns: **department_id** and **department_name**. The table contains three rows of data, each with **Edit**, **Copy**, and **Delete** actions.

	department_id	department_name
☐ Edit Copy Delete	101	IT
☐ Edit Copy Delete	102	Admin
☐ Edit Copy Delete	104	Finance

Server: 127.0.0.1 » Database: mydb » Table: employees

[Browse](#)
[Structure](#)
[SQL](#)
[Search](#)
[Insert](#)
[Export](#)
[Import](#)

✓ Showing rows 0 - 4 (5 total, Query took 0.0004 seconds.)

`SELECT * FROM `employees``

☐ Profiling
 [\[Edit inline \]](#)
[\[Edit \]](#)
[\[Explain SQL \]](#)
[\[Create PHP code \]](#)
[\[Refresh \]](#)

☐ Show all |
 Number of rows: 25 ▼
 Filter rows:
Sort by key: ▼

Extra options

		employee_id	name	department_id
☐	Edit Copy Delete	1	Jose	101
☐	Edit Copy Delete	2	Juan	102
☐	Edit Copy Delete	3	Maria	103
☐	Edit Copy Delete	4	Ipe	103
☐	Edit Copy Delete	5	Rommel	NULL

Inner Join

```
SELECT employees.name, departments.department_name
FROM employees
INNER JOIN departments
ON employees.department_id = departments.department_id;
```

Output

name	department_name
Jose	IT
Juan	Admin

Only rows with matching department_id in both tables are returned.

Left Join

```
SELECT employees.name, departments.department_name
FROM employees
LEFT JOIN departments
ON employees.department_id = departments.department_id;
```

Output

name	department_name
Jose	IT
Juan	Admin
Maria	NULL
Ipe	NULL
Rommel	NULL

All employees are shown, even if they don't belong to a department.

Right Join

```
SELECT employees.name, departments.department_name
FROM employees
RIGHT JOIN departments
ON employees.department_id = departments.department_id;
```

Output

name	department_name
Jose	IT
Juan	Admin
NULL	Finance

All departments are shown, even if no employee belongs to them.

Cross Join

```
SELECT * FROM employees CROSS JOIN departments;
```

Output

employee_id	name	department_id	department_id	department_name
1	Jose	101	101	IT
1	Jose	101	102	Admin
1	Jose	101	104	Finance
2	Juan	102	101	IT
2	Juan	102	102	Admin
2	Juan	102	104	Finance
3	Maria	103	101	IT
3	Maria	103	102	Admin
3	Maria	103	104	Finance
4	Ipe	103	101	IT
4	Ipe	103	102	Admin
4	Ipe	103	104	Finance
5	Rommel	NULL	101	IT
5	Rommel	NULL	102	Admin
5	Rommel	NULL	104	Finance

Each employee can have a possible of three departments

Self-join

Example



Example: Employees and Their Managers

Let's say we have a table called `employees` :

Table: `employees`

employee_id	name	manager_id
1	Alice	NULL
2	Bob	1
3	Charlie	1
4	Diana	2

Here:

- Alice is the top-level manager.
- Bob and Charlie report to Alice.
- Diana reports to Bob.

Requirement - display employee's manager/supervisor

employee_name	manager_name
Alice	NULL
Bob	Alice
Charlie	Alice
Diana	Bob

Script

```
SELECT
```

```

    e.name AS employee_name,
    m.name AS manager_name
FROM
    employees e
LEFT JOIN
    employees m ON e.manager_id = m.employee_id;

```

note. *self-join* is simply a **regular join** (like INNER JOIN, LEFT JOIN, etc.) where the **same table** is referenced twice. You distinguish the two instances of the table using **aliases**

FULL JOIN

- A **FULL JOIN** (also known as **FULL OUTER JOIN**) returns all records when there is a match in either left or right table. If there's no match, the result will contain NULL for missing values.
- MySQL doesn't support FULL JOIN directly, but you can simulate it using a combination of LEFT JOIN and RIGHT JOIN with UNION.

Example

Run this script

```

-- use test DATABASE
use test;
-- Create employees table
CREATE TABLE employees (
    emp_id INT,
    emp_name VARCHAR(50),
    dept_id INT
);

-- Create departments table
CREATE TABLE departments (
    dept_id INT,
    dept_name VARCHAR(50)
);

-- Insert sample data into employees
INSERT INTO employees VALUES
(1, 'Alice', 101),
(2, 'Bob', 102),
(3, 'Charlie', NULL);

-- Insert sample data into departments
INSERT INTO departments VALUES

```

```

(101, 'HR'),
(102, 'Finance'),
(103, 'IT');

-- Simulate FULL OUTER JOIN using UNION of LEFT and RIGHT JOIN
SELECT e.emp_id, e.emp_name, d.dept_name
FROM employees e
LEFT JOIN departments d ON e.dept_id = d.dept_id

UNION

SELECT e.emp_id, e.emp_name, d.dept_name
FROM employees e
RIGHT JOIN departments d ON e.dept_id = d.dept_id;

```

Output

1. Created two tables (employees and departments) with records
2. output of the join

emp_id	emp_name	dept_name
1	Alice	HR
2	Bob	Finance
3	Charlie	NULL
NULL	NULL	IT

Additional Exercises

Tables below

product

		product_id	product_name	price
☐	 Edit  Copy  Delete	1	Laptop	1000.00
☐	 Edit  Copy  Delete	2	Phone	500.00
☐	 Edit  Copy  Delete	3	Tablet	500.00
☐	 Edit  Copy  Delete	4	Keyboard	100.00
☐	 Edit  Copy  Delete	5	Mouse	50.00

Sale

		sale_id	product_id	quantity	sale_date
☐	 Edit  Copy  Delete	101	1	2	2023-08-01
☐	 Edit  Copy  Delete	102	2	1	2023-08-02
☐	 Edit  Copy  Delete	103	2	1	2023-08-02
☐	 Edit  Copy  Delete	104	3	1	2023-08-02
☐	 Edit  Copy  Delete	105	4	1	2023-08-02
☐	 Edit  Copy  Delete	106	5	3	2023-08-03

Full Script with table creation and record creation

```
-- use JoinDB
use JoinDB;
-- Create product table
CREATE TABLE product (
  product_id INT PRIMARY KEY,
  product_name VARCHAR(50),
  price DECIMAL(10, 2)
);

-- Create sale table
CREATE TABLE sale (
  sale_id INT PRIMARY KEY,
  product_id INT,
  quantity INT,
  sale_date DATE,
  FOREIGN KEY (product_id) REFERENCES product(product_id)
);

-- Insert sample data into product
INSERT INTO product (product_id, product_name, price) VALUES
```



```

(1, 'Laptop', 1000.00),
(2, 'Phone', 500.00),
(3, 'Tablet', 500.00),
(4, 'Keyboard', 100.00),
(5, 'Mouse', 50.00);

-- Insert sample data into sale
INSERT INTO sale (sale_id, product_id, quantity, sale_date) VALUES
(101, 1, 2, '2023-08-01'),
(102, 2, 1, '2023-08-02'),
(103, 2, 1, '2023-08-02'),
(104, 3, 1, '2023-08-02'),
(105, 4, 1, '2023-08-02'),
(106, 5, 3, '2023-08-03'); -- Note: product_id 5 doesn't exist in product

-- LEFT JOIN: All products, with matching sales if available
SELECT
    p.product_id, p.product_name, s.sale_id, s.quantity
FROM
    product p
LEFT JOIN
    sale s ON p.product_id = s.product_id;

-- RIGHT JOIN: All sales, with matching product info if available
SELECT
    p.product_id, p.product_name, s.sale_id, s.quantity
FROM
    product p
RIGHT JOIN
    sale s ON p.product_id = s.product_id;

-- FULL JOIN: All products and all sales, matched where possible
SELECT
    p.product_id, p.product_name, s.sale_id, s.quantity
FROM
    product p
FULL JOIN
    sale s ON p.product_id = s.product_id;

-- CROSS JOIN: Every combination of product and sale
SELECT
    p.product_name, s.sale_id, s.quantity
FROM
    product p
CROSS JOIN
    sale s;

-- SELF JOIN: Compare products to each other

```

```
SELECT
    a.product_name AS ProductA, b.product_name AS ProductB
FROM
    product a
JOIN
    product b ON a.product_id < b.product_id;
```

Issue the following command. What will be the output?

```
-- LEFT JOIN: All products, with matching sales if available
SELECT
    p.product_id, p.product_name, s.sale_id, s.quantity
FROM
    product p
LEFT JOIN
    sale s ON p.product_id = s.product_id;

-- RIGHT JOIN: All sales, with matching product info if available
SELECT
    p.product_id, p.product_name, s.sale_id, s.quantity
FROM
    product p
RIGHT JOIN
    sale s ON p.product_id = s.product_id;

-- FULL JOIN: All products and all sales, matched where possible
SELECT
    p.product_id, p.product_name, s.sale_id, s.quantity
FROM
    product p
FULL JOIN
    sale s ON p.product_id = s.product_id;

-- CROSS JOIN: Every combination of product and sale
SELECT
    p.product_name, s.sale_id, s.quantity
FROM
    product p
CROSS JOIN
    sale s;

-- SELF JOIN: Compare products to each other
SELECT
    a.product_name AS ProductA, b.product_name AS ProductB
```

```
FROM
    product a
JOIN
    product b ON a.product_id = b.product_id;
```

Joining three tables

Issue the script below

```
-- use db
use JOINDB;
-- Step 1: Create tables
CREATE TABLE departments (
    id INT PRIMARY KEY,
    department_name VARCHAR(50)
);

CREATE TABLE employees (
    id INT PRIMARY KEY,
    name VARCHAR(50),
    department_id INT,
    FOREIGN KEY (department_id) REFERENCES departments(id)
);

CREATE TABLE salaries (
    employee_id INT,
    salary DECIMAL(10, 2),
    FOREIGN KEY (employee_id) REFERENCES employees(id)
);

-- Step 2: Insert sample data
INSERT INTO departments (id, department_name) VALUES
(1, 'HR'),
(2, 'Finance'),
(3, 'Engineering'),
(4, 'IT'),
(5, 'Admin');

INSERT INTO employees (id, name, department_id) VALUES
(101, 'Alice', 1),
(102, 'Bob', 2),
(103, 'Charlie', 3),
(104, 'Juan', 4);
```

```
INSERT INTO salaries (employee_id, salary) VALUES
(101, 55000.00),
(102, 62000.00),
(103, 75000.00);
```

-- Step 3: Join all three tables

```
SELECT
    e.name AS employee_name,
    d.department_name,
    s.salary
FROM
    employees e
JOIN
    departments d ON e.department_id = d.id
JOIN
    salaries s ON e.id = s.employee_id;
```

Output for the join

employee_name	department_name	salary
Alice	HR	55000.00
Bob	Finance	62000.00
Charlie	Engineering	75000.00

Issue the ff: and analyze

```
-- LEFT JOIN: Get all employees, even if they don't have a salary record
SELECT
    e.name AS employee_name,
    d.department_name,
    s.salary
FROM
    employees e
LEFT JOIN
    departments d ON e.department_id = d.id
LEFT JOIN
    salaries s ON e.id = s.employee_id;
```

What It Does:

- Includes **all employees**, even if they don't have a matching department or salary.
- If there's no match, department_name or salary will be NULL

Isse the script below

```
-- RIGHT JOIN: Get all salary records, even if the employee is missing
SELECT
    e.name AS employee_name,
    d.department_name,
    s.salary
FROM
    salaries s
RIGHT JOIN
    employees e ON s.employee_id = e.id
RIGHT JOIN
    departments d ON e.department_id = d.id;
```

What It Does:

- Includes **all departments**, even if no employees are assigned.
- Also includes **all employees**, even if they don't have a salary.
- If there's no match, salary will be NULL.

***** END *****